tmux documentation

§08 — Recommendation

Revision: 1 **Last modified:** 2026-05-22T07:20:02Z **Authority:** vasic-digital tmux project (research-only) **Maintainer:** milosvasic **Scope:** Synthesis — should we port the vasic-digital tmux project to Termux on Android?

TL;DR

Conditional YES. A Termux port is technically feasible and would deliver real operator value. Recommended sequencing: **do NOT port now**; defer until v1.0.9 lands and the test harness for nezha-Linux is hardened. Then add Termux as the third platform (§11.4.81 cross-platform-parity branch) in a focused PWU. Effort: medium (≈ 2-3 weeks of subagent-driven work). Maintenance: medium-ongoing.

Decision matrix

Dimension	Assessment	Notes
Build complexity	LOW-MEDIUM	tmux + libevent + jemalloc all build cleanly on bionic (Termux's own package proves it). Go cross-compile is well-trodden territory. `setup.sh` extension is ~50 LOC.
Isolation primitives	MEDIUM	Lose cgroup features. Gain `RLIMIT_AS` enforcement vs macOS. Honest-gap citation table is clean.
End-user UX	MEDIUM	Touch keyboard is awkward but workable (Vol-Down emulates Ctrl, extra-keys row). Background suspension is the biggest pain point — requires operator wake-lock dance + battery-opt disable.
Test coverage	MEDIUM	SSH-driven harness is straightforward (same shape as nezha bridge). New failure-mode classes (Doze, Phantom Process Killer, lmkd) require explicit topology detection. ~200 LOC new test scaffolding.
Maintenance burden	MEDIUM-ONGOING	Android version drift every 12 months. Termux package cadence is currently slower than 2019-2022 peak. Need at least one phone in regular CI rotation.
Operator demand	UNKNOWN	This research was prompted by a "could we" question, not a count of requests. **Validation needed**: ask operators whether they'd actually use it.

Recommendation rationale

A Termux port is not technically expensive. The hard parts of `tmx` (per-session isolation, anti-bluff testing, multi-platform parity) are already designed to extend cleanly via §11.4.81. The new mechanical work is:

1. ~50 LOC in `install_deps.sh` + `setup.sh` for Termux detection and package install.
2. A Termux branch in `tmx.template` reusing the existing rlimit wrapper.
3. A `Port 8022` flag in `tmx-ssh-install.sh`.
4. A documented operator-prep checklist (wake-lock, battery-opt, Phantom killer).
5. ~200 LOC of test-harness adaptation per §07.
6. Documentation: new `docs/guides/tmx-on-termux.md` + per-script companion guides.

What does NOT need work: the Go state daemon, the shell-init script, the SSH dispatcher, tmux itself, jemalloc preload, the project's verify/test/Challenge infrastructure.

The expensive part is **maintenance**: we'd commit to running Termux tests on every release, keeping pace with Android API changes, and supporting end users on a less-predictable OS.

Conditions for proceeding

The conditional "yes" depends on:

1. **v1.0.9 ships clean on Linux + macOS first.** Adding a third platform mid-feature-development is §11.4.42-iteration-discipline violation territory.
2. **Operator demand is validated.** A 1-question Issues.md poll: "Would you use `tmx` on your Android phone via Termux?" If <30% yes, defer the port until demand grows.
3. **At least one test device dedicated.** Either a developer's spare phone or a 2-3 year old phone we keep plugged into the test bench. Termux on an emulator is insufficient for the §06 WILL-DEGRADE items.
4. **Tailscale or equivalent established for remote access.** Without it, dev-loop friction (laptop ⍰ phone test cycle) is prohibitive on cellular networks.
5. **One operator volunteers to be the Termux-using-end-user beta tester.** §11.4 covenant demands end-user-validation; we need a real end user who actually uses tmux on Android.

If all five hold: ship a v1.1.x feature release adding Termux support. If any fail: defer.

What would the v1.1.x scope look like?

PWU	Scope	Estimate
T1	`install_deps.sh` + `setup.sh` Termux branch	2 days
T2	`tmx.template` Termux dispatch + RLIMIT_AS branch	2 days
T3	`tmx-ssh-install.sh --port 8022` autodetect	1 day
T4	`tmx doctor` Termux-environment probe (wake-lock, battery-opt, Phantom killer)	2 days
T5	Test harness: `scripts/test_termux.sh` + per-test Termux branches	4 days
T6	Pre-build gates + paired §1.1 mutations for Termux	2 days
T7		2 days

PWU	Scope	Estimate
	Docs: `docs/guides/tmx-on-termux.md`, operator setup checklist, troubleshooting	
T8	First real-device shake-out cycle (10× full retest per §11.4.50)	1 day
T9	§11.4.40 full-suite retest on Linux+macOS+Termux before tagging	1 day
	Total	**~17 working days**

That fits comfortably into a 3-week iteration with subagent parallelism per §11.4.58 + §11.4.70. PWUs T1-T3 are parallel; T4-T5 are parallel; T6-T7 are parallel; T8-T9 serial.

What we do NOT recommend

- **A "minimal viable port" that skips lmkd / Doze handling.** That is exactly the §11.4 "tests green but feature broken for end user" failure mode the covenant exists to prevent. If we port, we port with the operator-prep checklist + the `tmx doctor` probe + the SKIP-with-reason vocabulary for unavoidable Android killers. Otherwise we ship a feature that "works on my screen-on dev phone" and breaks for everyone else.
- **A Play Store distribution.** Termux itself isn't on Play Store (frozen 2022). We have no business trying to ship a tmux wrapper there.
- **A rooted-cgroup-v2 default path.** Document the rooted opt-in for power users, but the supported default is unrooted Termux with rlimit.
- **Cross-host state sync.** v1.0.9 explicitly excludes this (spec §3 non-goal). Adding it to support "my work session on phone matches my work session on laptop" doubles the design surface. Stay focused.

Open questions for operator approval

Per §11.4.66 (blocker-resolution-interactive-clarification), the decisions that ultimately need operator sign-off:

1. **Go / no-go on the v1.1.x port itself.** This document recommends conditional-yes with the five conditions above. Operator may say "go ahead anyway" or "defer indefinitely."
2. **Default port 8022 vs alternate.** 8022 is Termux convention; we recommend keeping it.
3. **Wake-lock policy.** Default to operator-runs-it-themselves (notice), or wrapper auto-acquires?
4. **Whether to publish a Termux-targeted recipe via `termux-packages` upstream.** Path (c) in §02 — currently not recommended, but operator may differ.
5. **CI investment.** Buy a dedicated test phone (~$300-500) and integrate it into the nezha-bridge pattern, or rely on emulator-only smoke + operator-driven full-sweep?

Each is a closed-set decision well-suited to AskUserQuestion if operator chooses to advance.

Recommendation summary

Defer until v1.0.9 ships and operator demand is validated. Then a focused 3-week PWU lands Termux as a first-class third platform.

The technical risk is low. The maintenance commitment is real but bounded. The honest-gap citation is clean (we don't pretend Termux gives us what cgroups give us, and we explicitly document the Android-killer caveats per §11.4.6).

If at any point operator demand crystallises into a hard requirement ("this MUST work on Termux"), the work is well-understood and ready to begin.

Sources

(synthesised from §01-§07; no new sources cited here — see prior files for evidence)